

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

- CLI To-Do List Manager - Technical Specification & Architecture Document 3**
 - 1. Executive Summary & Architecture Overview 3
 - 1.1 Executive Brief 3
 - 1.2 Maturity Assessment 3
 - 1.3 Technical Stack 3
 - 1.4 Architectural Constraints 3
 - 1.5 Critical Dependencies 3
 - 2. Architecture Workflows 4
 - 2.1 Project Implementation Traceability Map 6
 - 2.2 CLI Task Management Workflow 6
 - 2.3 CLI To-Do Data Model 7
 - 2.4 Task Operation Sequence 8
 - 3. Detailed Technical Specifications & Business Rules 9
 - 3.1 Requirements Traceability 9
 - 3.2 Security Rules 12
 - 3.3 Data Models 13
 - 4. Project Governance & Structural Gaps 13
 - 4.1 Structural Gaps 13
 - 4.2 Remediation & Workflow 13
 - 5. Technical & Domain Glossary (Terminology Reference) 13

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

A Python-based CLI To-Do Manager utilizing a local JSON file for persistent storage. The system implements a service-oriented architecture separating CLI dispatch, business logic, and file I/O to manage task lifecycles including creation, completion, and removal.

1.2 Maturity Assessment

The project is structurally sound with a high health index and a clear execution roadmap. While formal acceptance criteria and explicit security constraints are missing, the presence of independent test cases for each user story provides sufficient validation paths. The project is READY for execution.

1.3 Technical Stack

- Python
- pyproject.toml

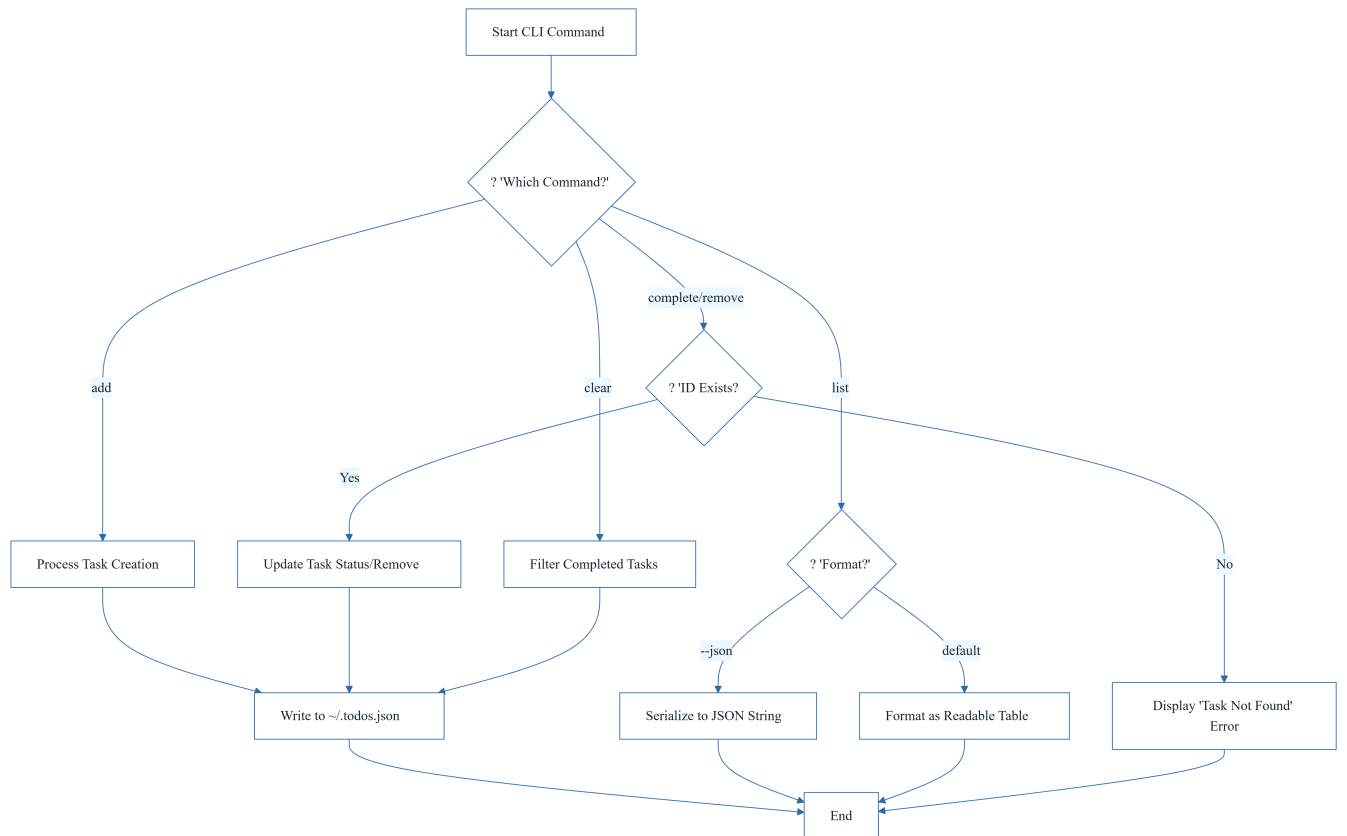
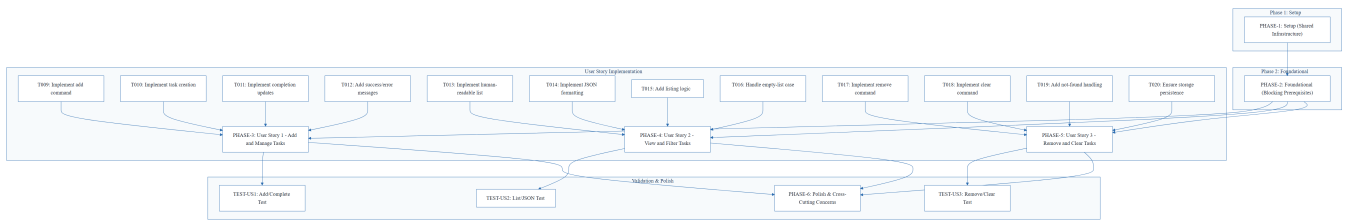
1.4 Architectural Constraints

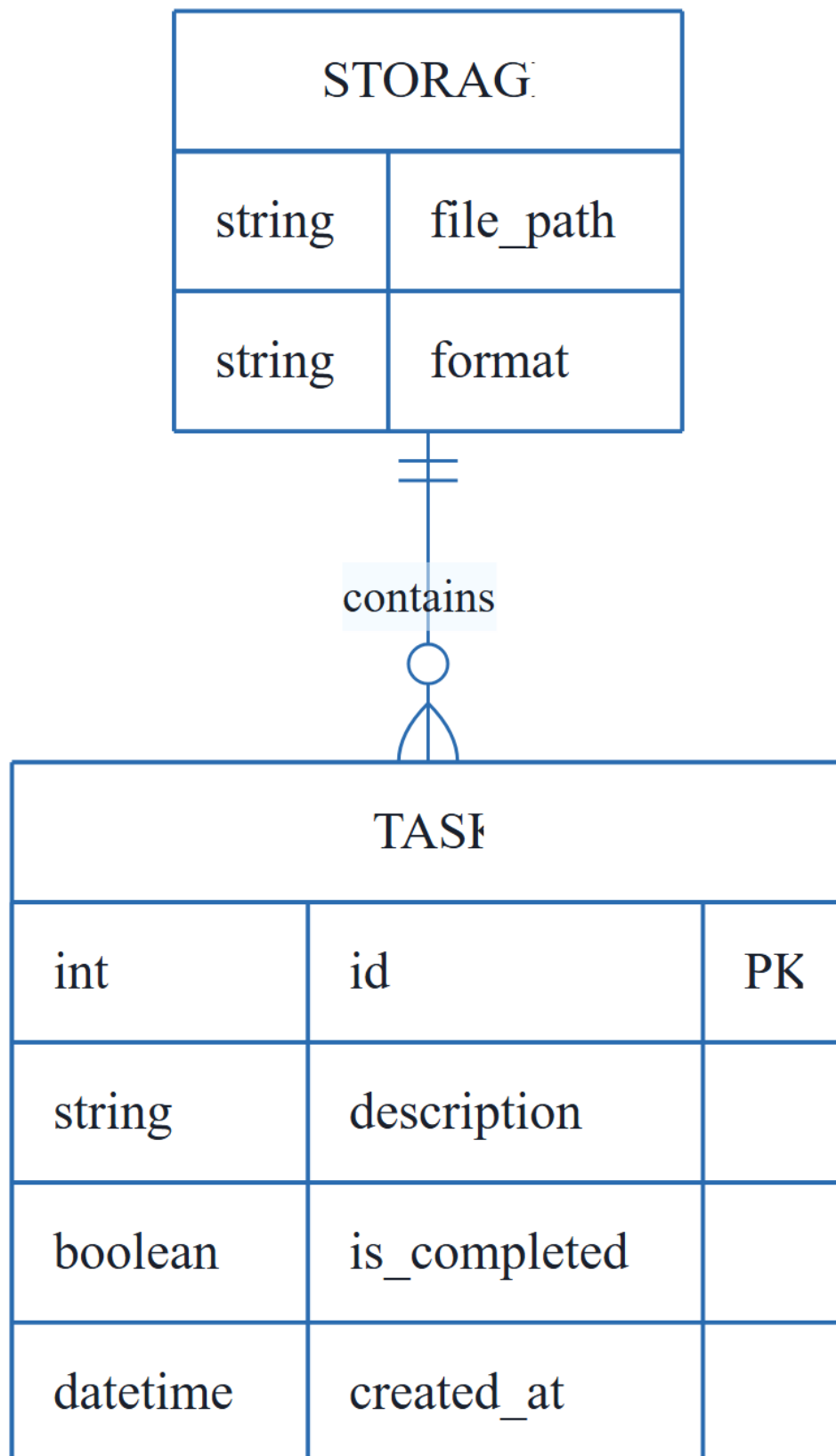
- Storage must be persisted in `~/todos.json`.
- Sequential ID assignment for task creation.
- Mandatory population of `created_at` timestamps.
- Strict execution order: Phase 2 (Foundational) must be complete before any User Story implementation.
- Output must support both human-readable and parseable JSON formats via `--json` flag.

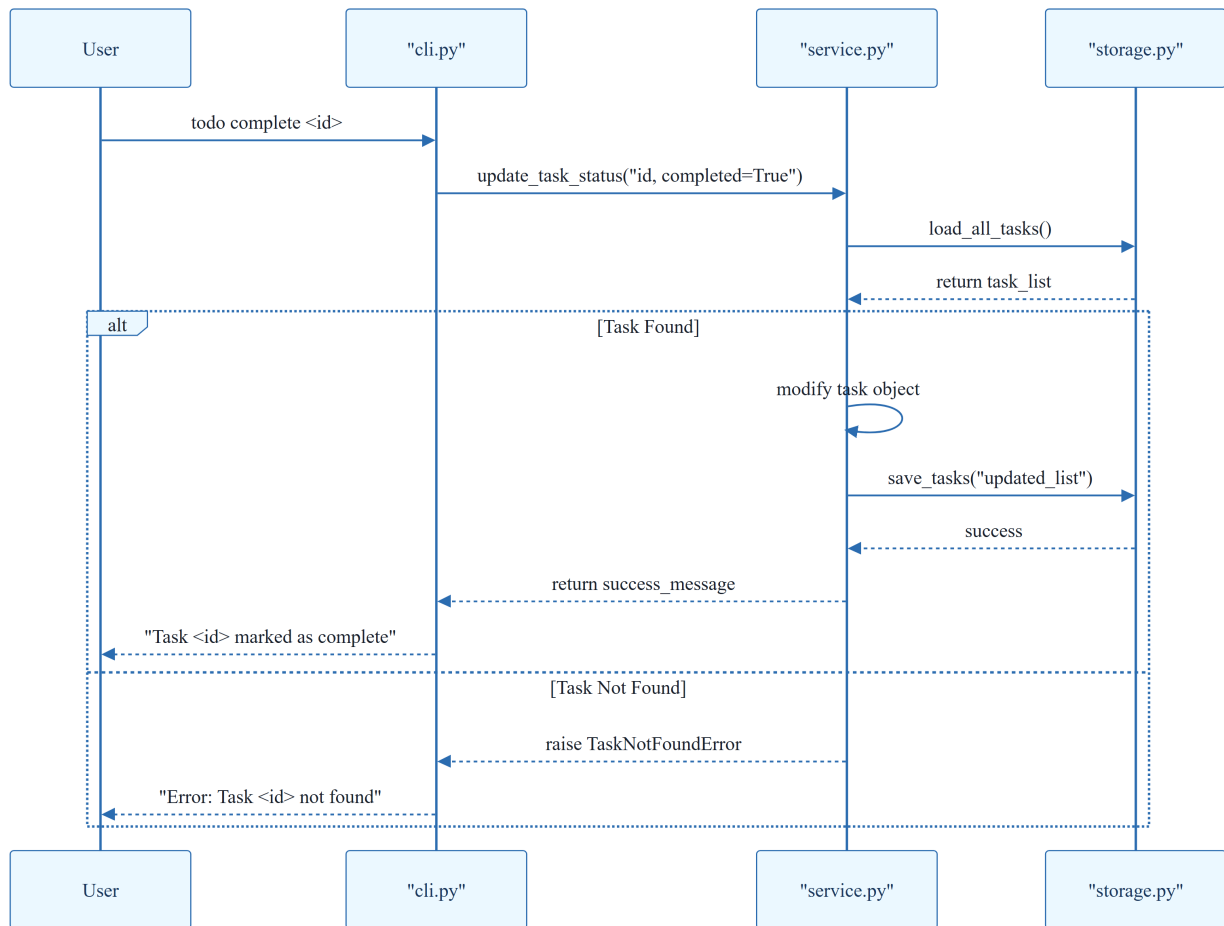
1.5 Critical Dependencies

- JSON storage path resolution and file I/O helpers in `storage.py`.
- Task entity serialization helpers in `models.py`.
- Sequential dependency: Setup $\rightarrow$ Foundational $\rightarrow$ User Stories $\rightarrow$ Polish.
- Internal data integrity: Task ID validity for remove and complete operations.
- Environment: Python runtime with `pyproject.toml` entry points.

2. Architecture Workflows



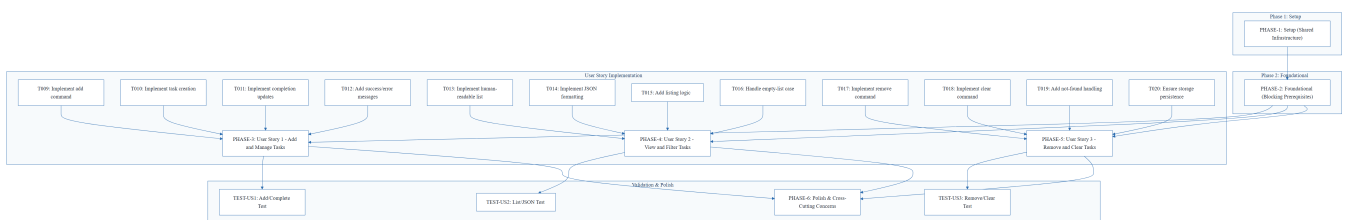




& Visual Diagrams

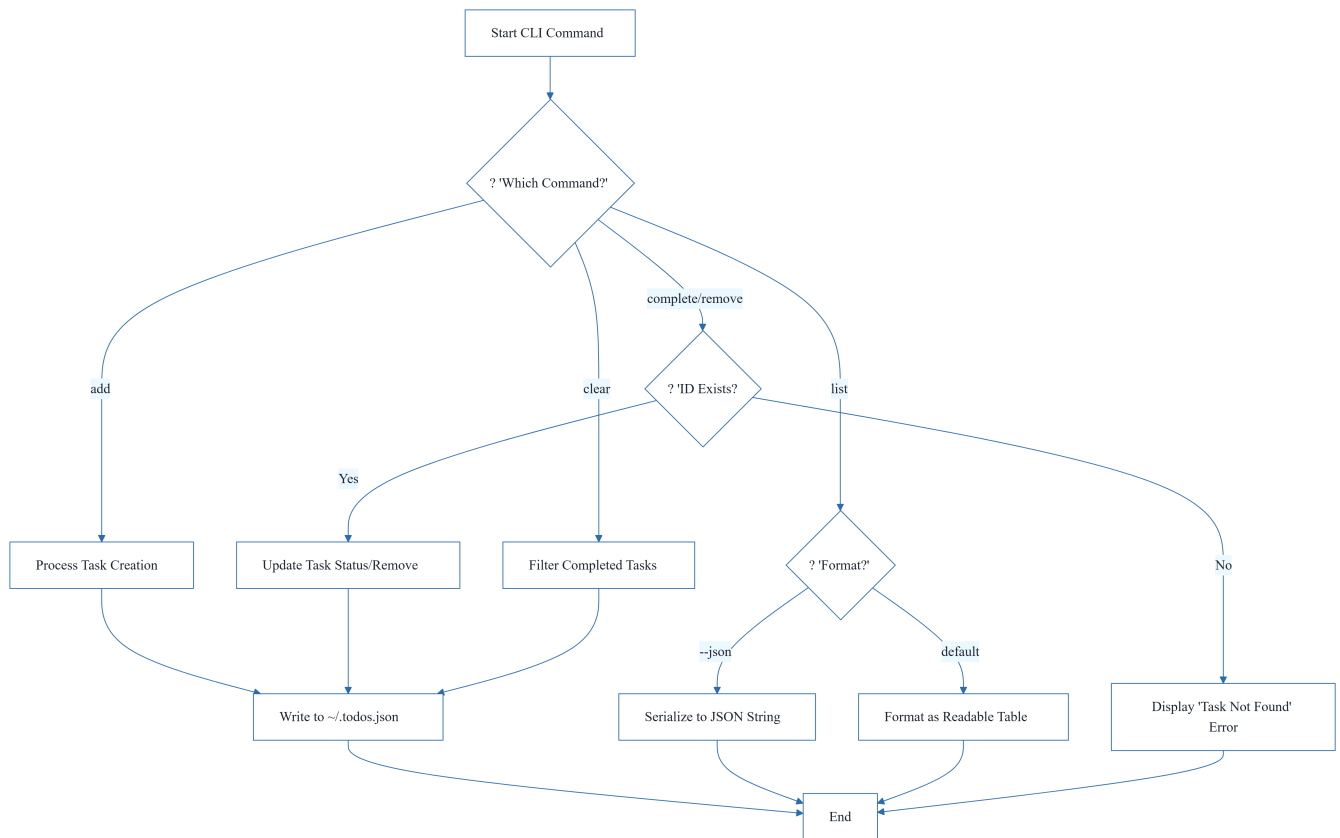
2.1 Project Implementation Traceability Map

Maps the dependency flow from project setup through foundational work to specific user story implementation and final polish.



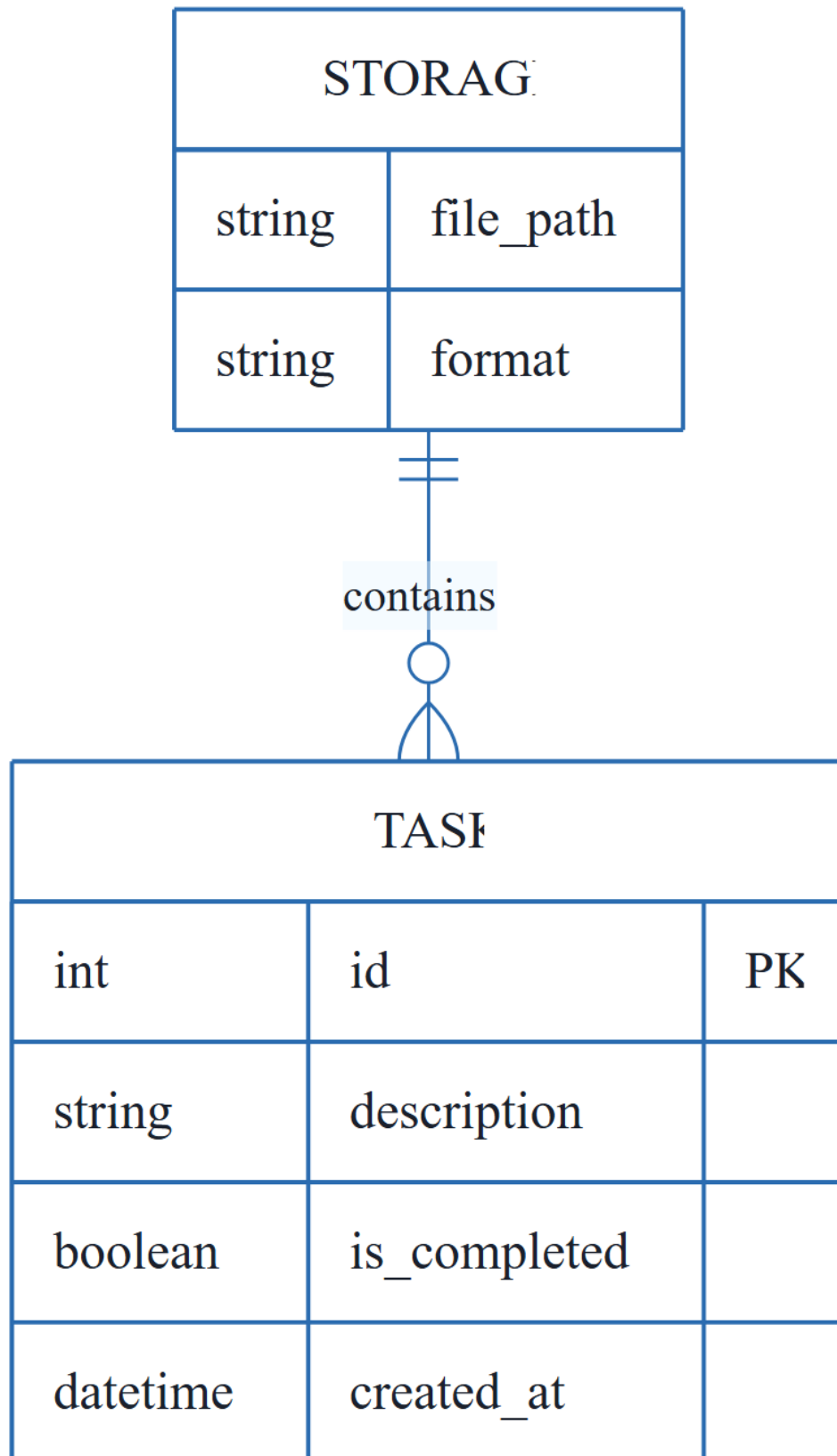
2.2 CLI Task Management Workflow

Business logic flow for managing tasks via the CLI, including decision points for output formatting and error handling.



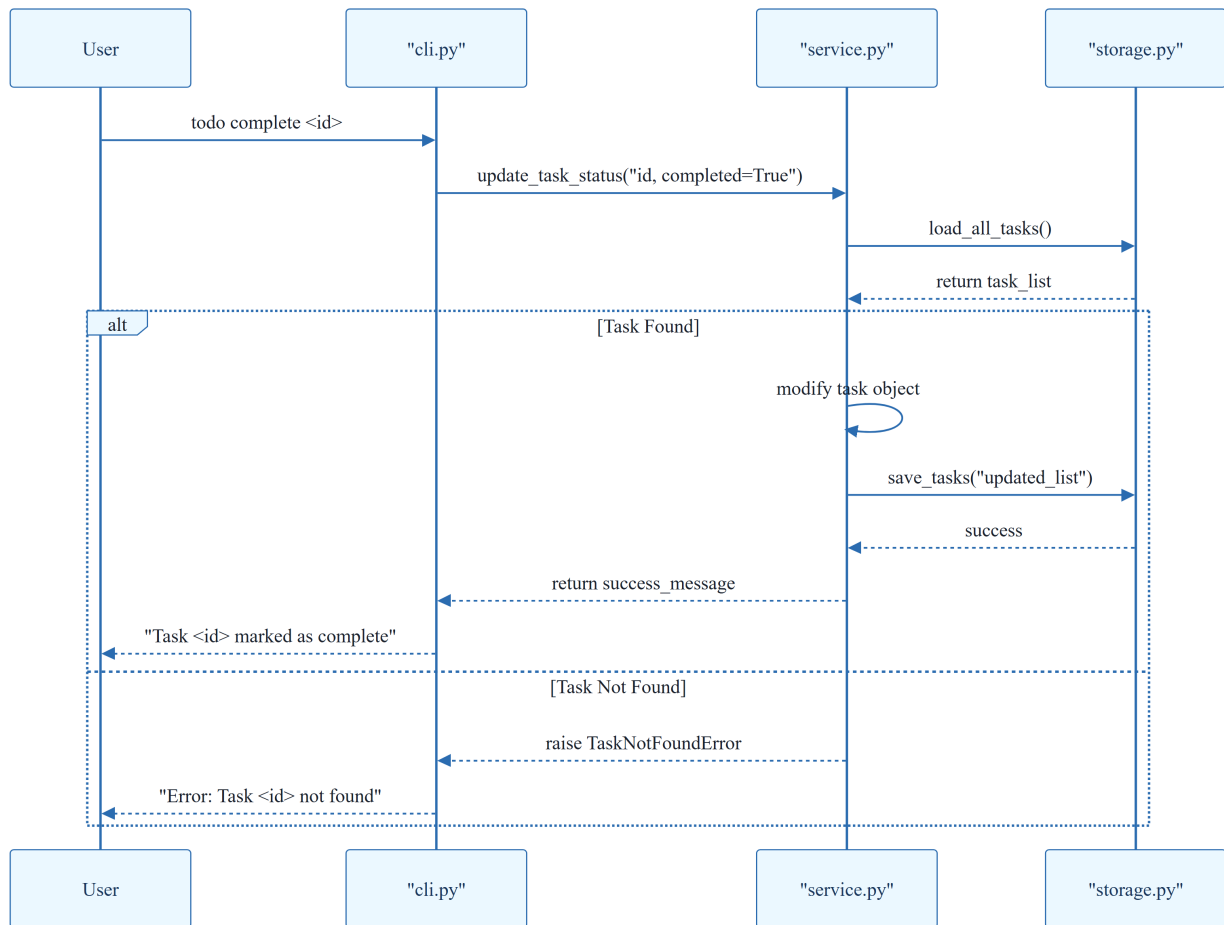
2.3 CLI To-Do Data Model

The logical data structure for the To-Do manager stored in JSON.



2.4 Task Operation Sequence

Interaction flow between the CLI layer, Service layer, and Storage layer for a typical task update.



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Type	Description	Status	Source Section
T001	task	Create the Python package skeleton in <code>src/todo_manager/init.py</code> , <code>main.py</code> , <code>cli.py</code> , <code>models.py</code> , <code>storage.py</code> , and <code>service.py</code>	completed	Phase 1: Setup (Shared Infrastructure)
T002	task	Create the test directory structure in <code>tests/unit/</code> and <code>tests/integration/</code>	completed	Phase 1: Setup (Shared Infrastructure)

ID	Type	Description	Status	Source Section
T003	task	Add project metadata and tooling entry points in <code>pyproject.toml</code>	completed	Phase 1: Setup (Shared Infrastructure)
T004	task	Implement task entity definitions and serialization helpers in <code>src/todo_manager/models.py</code>	completed	Phase 2: Foundational (Blocking Prerequisites)
T005	task	Implement JSON storage path resolution and file I/O helpers in <code>src/todo_manager/storage.py</code>	completed	Phase 2: Foundational (Blocking Prerequisites)
T006	task	Implement shared command-line parsing and top-level dispatch in <code>src/todo_manager/cli.py</code>	completed	Phase 2: Foundational (Blocking Prerequisites)
T007	task	Implement shared service-layer error handling and task collection utilities in <code>src/todo_manager/service.py</code>	completed	Phase 2: Foundational (Blocking Prerequisites)
T008	task	Define module entry behavior for python <code>-m todomanager</code> in <code>src/todomanager/main.py</code>	completed	Phase 2: Foundational (Blocking Prerequisites)
T009	task	Implement the <code>add</code> command flow in <code>src/todomanager/cli.py</code> and <code>src/todomanager/service.py</code>	completed	Implementation for User Story 1
T010	task	Implement task creation, sequential ID assignment, and <code>created_at</code> population in <code>src/todo_manager/models.py</code>	completed	Implementation for User Story 1
T011	task	Implement completion updates for existing tasks in <code>src/todo_manager/service.py</code>	completed	Implementation for User Story 1

ID	Type	Description	Status	Source Section
T012	task	Add user-facing success and error messages for add and complete operations in <code>src/todo_manager/cli.py</code>	completed	Implementation for User Story 1
T013	task	Implement human-readable task listing output in <code>src/todo_manager/cli.py</code>	pending	Implementation for User Story 2
T014	task	Implement <code>--json</code> output formatting in <code>src/todomanager/cli.py</code> using the task serialization helpers in <code>src/todomanager/models.py</code>	pending	Implementation for User Story 2
T015	task	Add listing logic that preserves task order and status fields in <code>src/todo_manager/service.py</code>	completed	Implementation for User Story 2
T016	task	Handle the empty-list case with a clear message in <code>src/todo_manager/cli.py</code>	completed	Implementation for User Story 2
T017	task	Implement the <code>remove</code> command flow in <code>src/todomanager/cli.py</code> and <code>src/todo_manager/service.py</code>	pending	Implementation for User Story 3
T018	task	Implement the <code>clear</code> command flow for removing completed tasks in <code>src/todo_manager/service.py</code>	pending	Implementation for User Story 3
T019	task	Add not-found handling for invalid task IDs in <code>src/todo_manager/cli.py</code>	pending	Implementation for User Story 3
T020	task	Ensure storage writes persist removals and clears safely in <code>src/todo_manager/storage.py</code>	pending	Implementation for User Story 3
T021	task	Document the CLI usage and storage behavior in <code>README.md</code>	pending	Phase 6: Polish & Cross-Cutting Concerns

ID	Type	Description	Status	Source Section
T022	task	Add quickstart verification notes and examples in specs/001-cli-todo-manager/quickstart.md	pending	Phase 6: Polish & Cross-Cutting Concerns
T023	task	Run a manual smoke test of add, list, complete, remove, and clear against ~/.todos.json	pending	Phase 6: Polish & Cross-Cutting Concerns
T024	task	Verify linting and formatting expectations for the source files in src/todo_manager/	pending	Phase 6: Polish & Cross-Cutting Concerns
T025	task	Confirm the generated JSON output remains parseable for the todo list --json path	pending	Phase 6: Polish & Cross-Cutting Concerns
TEST-US1	test_case	Run <code>todo add "Task description"</code> and <code>todo complete</code> , then confirm the stored task list reflects the new task and its completion status.	N/A	Phase 3: User Story 1 - Add and Manage Tasks (Priority: P1) ■ MVP
TEST-US2	test_case	Run <code>todo list --json</code> and confirm the output is readable or parseable JSON while showing the full task collection.	N/A	Phase 4: User Story 2 - View and Filter Tasks (Priority: P2)
TEST-US3	test_case	Run <code>todo remove</code> and <code>todo clear</code> , then confirm the targeted task or completed tasks are removed while other tasks remain intact.	N/A	Phase 5: User Story 3 - Remove and Clear Tasks (Priority: P3)

3.2 Security Rules

- No explicit security constraints provided in source.
- Remediation: Implement basic input sanitization for task descriptions to prevent injection or formatting corruption in the JSON store.

3.3 Data Models

- **Task Entity:**
 - `id` (Integer, PK): Sequential unique identifier.
 - `description` (String): Text content of the task.
 - `is_completed` (Boolean): Completion status.
 - `created_at` (DateTime): Timestamp of creation.
- **Storage:**
 - `file_path`: `~/.todos.json`.
 - `format`: JSON.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Acceptance Criteria	MEDIUM	While 'Independent Tests' are provided, formal acceptance criteria for each task would improve validation.
Security & Performance Constraints	LOW	The project is a simple CLI tool, but constraints on file size or input sanitization could be added.
Open Questions & Uncertainties	LOW	No open questions were listed in the source document.

4.2 Remediation & Workflow

1. **Validation:** Integrate the TEST-USx cases as formal acceptance criteria for each User Story.
2. **Hardening:** Add a validation layer in `service.py` to ensure `id` inputs are positive integers.
3. **Documentation:** Ensure T021 and T022 are completed to provide a clear onboarding path for new developers.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-2	A synchronization gate ensuring all blocking infrastructure is verified before parallel feature development commences.

Term	Category	Context Anchor	Project Definition
Foundational	TECHNICAL_STACK	PHASE-2	The core shared infrastructure layer that provides essential services and blocking prerequisites for all subsequent user stories.
Goal	BUSINESS_DOMAIN	PHASE-3	The primary functional objective a user must achieve within a specific feature set.
ID	TECHNICAL_STACK	T010	A sequential numeric unique identifier assigned to each task entity during creation.
JSON	TECHNICAL_STACK	T005	The lightweight data-interchange format used for persistent storage and scriptable output.
MVP	BUSINESS_DOMAIN	PHASE-3	The minimum viable product consisting of the first priority user story to enable basic task creation and completion.
Organization	BUSINESS_DOMAIN	Tasks: CLI To-Do List Manager	The structural grouping of implementation units by user story to ensure independent testability.
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The set of mandatory design documents and specifications required before implementation begins.
README	TECHNICAL_STACK	T021	The primary documentation file detailing command-line usage and storage behavior.
Setup	TECHNICAL_STACK	PHASE-1	The initial project bootstrapping phase involving package skeleton creation and tooling configuration.
T016	TECHNICAL_STACK	T016	The specific implementation unit responsible for handling empty collection states with user-facing messages.
Tests	TECHNICAL_STACK	T002	The verification suite comprising unit and integration directory structures for validating system behavior.

Term	Category	Context Anchor	Project Definition
population in	TECHNICAL_STACK	T010	The process of assigning a timestamp to the creation date field of a task entity.
todo clear	BUSINESS_DOMAIN	TEST-US3	The operational command that purges all tasks marked as finished from the persistent store.
todo list	BUSINESS_DOMAIN	TEST-US2	The operational command that retrieves and displays the current collection of tasks in human-readable or machine-parseable formats.
■■ CRITICAL	TECHNICAL_STACK	PHASE-2	A high-severity blocking constraint that prohibits the start of any feature work until the current phase is fully validated.